VMM — Vulnerability Management Module

VMM — Vulnerability Management Module

OOF® Origin Open Foundation®

Independent Methodological Authority

Architecture: SLGA™ — Software Lifecycle Governance Architecture

Parent Standard: Software Security Standard (SSS)

Category: Governance & Enforcement

Subcategory: Software Lifecycle Governance

Type: Core Module

Governed Space: Software Vulnerability Governance

Module ID: SLGA-SSS-VMM-004

Origin ID: OOF.CM.SLGA.06.04.01

Version: 1.0

Status: Canonical · Open Standard

Effective Date: 13.10.2026

Compatibility: OOF® Methodology OS™ · Modular OOF® Architecture Ecosystem

GIE-Compatible: Yes

AI-Readable: Yes

Authority: OOF® — OriginOpen® Foundation

Protection: MIP® — Methodological Intellectual Property

Canonical Language: English (UCL™)

Canonical Definition

The Vulnerability Management Module (VMM) governs the identification,

recognition, confirmation, classification, contextual assessment, prioritization, treatment, remediation verification, determination, and lifecycle continuity of software vulnerabilities.

VMM establishes the methodological conditions under which a reported

weakness, security finding, vulnerability advisory, automated detection, observed failure, or suspected exploitable condition can be recognized as an identifiable, contextually applicable, evidence-supported software vulnerability rather than being assumed to constitute a confirmed vulnerability solely because a scanner, advisory, AI agent, or external source has reported it.

It governs the relationships among vulnerability origin, affected software

identity, protected assets, relevant security requirements, threat exposure, vulnerability existence, exploitability, accessibility, severity, operational impact, treatment decisions, corrective changes, remediation evidence, verification scope, residual vulnerability conditions, and vulnerability state continuity.

VMM distinguishes vulnerability reporting from confirmation, severity from

operational priority, theoretical exploitability from demonstrated exploitability, remediation implementation from verified remediation, administrative closure from technical resolution, and vulnerability resolution from final security conformity or residual risk acceptance.

It applies to vulnerabilities affecting software created, integrated,

maintained, modified, or operated by humans, organizations, automated development systems, AI coding assistants, autonomous software agents, and software infrastructure mechanisms.

VMM is technology-independent and does not prescribe a particular vulnerability scanner, severity scoring framework, vulnerability database, patch management system, security testing platform, or remediation methodology.

Its principal output is:

Governed Software Vulnerability State

Its foundational distinction is:

Vulnerability Closure ≠ Verified Vulnerability Remediation.

Operational Role

The operational role of VMM is:

To ensure that software vulnerability reports, confirmed weaknesses, severity assessments, exploitability conditions, treatment priorities, remediation actions, verification evidence, unresolved conditions, and lifecycle changes remain identifiable, contextually applicable, technically assessable, evidence-supported, and traceable to the actual software artifact and configuration.

VMM governs the complete vulnerability lifecycle from initial

identification to competent resolution determination or explicitly governed unresolved status.

It does not independently establish final software security conformity, accept residual security risk, or authorize software release or deployment.

Operational Question

How can reported or suspected software vulnerabilities be competently identified, confirmed, assessed, prioritized, treated, and verified against the actual affected software and operating conditions so that scanner findings, vulnerability severity, implemented patches, closed tickets, or favorable automated reports are not mistaken for demonstrated vulnerability resolution?

Position in Architecture Governance Flow

The SLGA™ governance flow is:

Software Requirements → Software Creation → Software Dependencies →

Software Configuration → Software Quality → Software Security → Software Release → Software Deployment → Software Change Management → Software Retirement

Cross-cutting:

Software Lifecycle Control

VMM operates within:

Software Security Standard (SSS)

It is the fourth Core Module of SSS.

The SSS module governance flow is:

Security Requirements → Security Exposure Assessment → Security Testing &

Control → Vulnerability Management → Security Risk & Conformity → Governed Software Security State

VMM governs:

Vulnerability Management

Its principal upstream references are:

SRQM — Security Requirements Module

Input: Governed Software Security Requirements State

SEAM — Security Exposure Assessment Module

Input: Governed Software Security Exposure State

STCM — Security Testing & Control Module

Input: Governed Software Security Testing & Control State

Additional references may include:

-

Software creation and artifact identity from SCS. -

Dependency identity and provenance from SDS. -

Build and configuration correspondence from SCFS. -

Relevant software defect information from SQS. -

Operational observations and security incident evidence where applicable.

Its principal output is:

Governed Software Vulnerability State

This output supports:

SRCM — Security Risk & Conformity Module

VMM may also return new findings to SRQM, SEAM, or STCM when a vulnerability reveals missing requirements, previously unrecognized exposure, or ineffective controls.

• Governed Space

Software Vulnerability Governance

This space governs:

-

Vulnerability governance object identity. -

Vulnerability report origin. -

Vulnerability identification. -

Vulnerability intake. -

Software artifact correspondence. -

Dependency correspondence. -

Configuration correspondence. -

Vulnerability evidence. -

Vulnerability confirmation. -

False-positive determination. -

Duplicate vulnerability identification. -

Vulnerability classification. -

Vulnerability severity. -

Vulnerability exploitability. -

Vulnerability exposure. -

Protected asset impact. -

Operational consequences. -

Vulnerability prioritization. -

Vulnerability ownership references. -

Vulnerability treatment. -

Remediation planning. -

Corrective action correspondence. -

Patch implementation. -

Dependency remediation. -

Configuration remediation. -

Compensating controls. -

Remediation testing. -

Remediation verification. -

Regression assessment. -

Residual vulnerability conditions. -

Vulnerability exceptions. -

Vulnerability deferral. -

Vulnerability reopening. -

Vulnerability closure eligibility. -

Vulnerability evidence gaps. -

Critical vulnerability governance gaps. -

Vulnerability uncertainty. -

Vulnerability conflicts. -

Vulnerability state determination. -

Vulnerability continuity. -

Vulnerability reassessment. -

Autonomous vulnerability management. -

Vulnerability governance outputs.

Vulnerability Governance Object

Every material VMM determination shall establish an identifiable Vulnerability

Governance Object.

The object represents a reported, suspected, confirmed, or previously

recognized software vulnerability associated with an identifiable software subject.

It should preserve:

Vulnerability Identity: A stable reference for the vulnerability.

Vulnerability Origin: The source of the report or finding.

Software Object: The affected component, application, service, interface,

or system.

Artifact Identity: The relevant software artifact or version.

Dependency Reference: The affected dependency where applicable.

Configuration: The relevant configuration conditions.

Security Requirement: The potentially affected protective requirement.

Exposure Reference: The relevant SEAM exposure conditions.

Testing Reference: The relevant STCM findings and evidence.

Vulnerability Description: The suspected or established weakness.

Confirmation State: The evidence-supported recognition status.

Severity: The assessed seriousness of the vulnerability.

Exploitability: The relevant exploitation conditions.

Impact: The potential or demonstrated consequences.

Treatment: The selected corrective or compensating approach.

Remediation Evidence: The evidence supporting corrective action.

Verification State: The recognized remediation verification condition.

Governance State: The overall vulnerability status.

A vulnerability record shall not be treated as confirmed merely because it

has been created by a scanner, external advisory, or autonomous agent.

Vulnerability Report ≠ Confirmed Software Vulnerability.

Vulnerability Governance Principles

VMM establishes the following principles.

Identifiable Vulnerability Subject: Every material finding must concern identifiable software.

Source Traceability: Vulnerability reports must preserve their origin.

Artifact Correspondence: Findings must remain connected to relevant software artifacts.

Contextual Applicability: Vulnerability relevance must be assessed against actual software conditions.

Confirmation Discipline: Reports must remain distinguishable from confirmed vulnerabilities.

Severity Separation: Severity must remain distinguishable from treatment priority.

Exploitability Separation: Theoretical exploitation must remain distinguishable from demonstrated exploitation.

Treatment Traceability: Corrective actions must remain connected to the vulnerability they address.

Remediation Verification: Implemented changes must not be mistaken for verified remediation.

Residual Condition Preservation: Unresolved vulnerability conditions must remain visible.

Closure Integrity: Administrative closure must not replace technical verification.

Authority Preservation: Technical assessment must remain distinct from authorized risk acceptance.

Lifecycle Continuity: Vulnerability history must remain traceable.

Reassessment: Material changes may invalidate previous determinations.

Vulnerability Management Purpose

VMM shall establish the purpose of material vulnerability management

activities.

Purposes may include:

-

Identifying software weaknesses. -

Confirming reported vulnerabilities. -

Assessing applicability. -

Determining exploitability. -

Evaluating severity and impact. -

Establishing treatment priority. -

Coordinating remediation. -

Verifying corrective actions. -

Preserving unresolved conditions. -

Supporting residual security risk evaluation.

A management activity shall remain connected to the vulnerability condition

it is intended to address.

Vulnerability Management Activity ≠ Vulnerability Resolution.

Vulnerability Management Scope

VMM shall establish the software scope affected by a vulnerability

determination.

Scope may concern:

-

A software function. -

A component. -

A dependency. -

An application. -

A service. -

An interface. -

An integrated system. -

An autonomous agent. -

A particular configuration. -

A deployment environment.

A vulnerability confirmed in one component shall not automatically

establish that every application using that component is exploitable.

Component Vulnerability ≠ Universal Application Exploitability.

Software Artifact Correspondence

VMM shall preserve the identity of the software artifact affected by a

vulnerability.

Relevant references may include:

-

Artifact identifier. -

Version. -

Build identity. -

Dependency composition. -

Configuration baseline. -

Runtime environment. -

Relevant deployment references.

A vulnerability determination concerning one artifact shall not

automatically establish the condition of another artifact.

Vulnerability State of Build A ≠ Vulnerability State of Build B.

Dependency Correspondence

VMM shall preserve relevant dependency relationships.

A vulnerability may affect:

-

A direct dependency. -

A transitive dependency. -

A shared library. -

A runtime component. -

A development tool. -

A build-time dependency. -

An external software service.

SDS governs dependency identity and lifecycle relationships.

VMM governs the vulnerability conditions associated with those dependencies.

Configuration Correspondence

VMM shall identify configuration conditions affecting vulnerability

relevance.

Examples include:

-

Enabled vulnerable functionality. -

Network exposure. -

Authentication requirements. -

Runtime privileges. -

Feature flags. -

Dependency settings. -

Security control configuration.

A vulnerability may be present in software while exploitation remains dependent on specific configuration conditions.

Vulnerability Presence ≠ Exploitability Under Every Configuration.

Vulnerability Report Origin

VMM shall preserve the origin of vulnerability reports.

Origins may include:

-

Automated scanners. -

Security researchers. -

Software vendors. -

Public vulnerability advisories. -

Internal testing. -

Penetration testing. -

Operational observations. -

Incident investigations. -

Developers. -

AI coding assistants. -

Autonomous security agents.

Report origin shall remain distinguishable from report validity.

Trusted Vulnerability Source ≠ Automatically Applicable Finding.

Vulnerability Intake

VMM shall govern the initial recognition of vulnerability information.

Intake should preserve:

-

Report identity. -

Source. -

Receipt date. -

Affected software reference. -

Description. -

Initial severity. -

Available evidence. -

Known uncertainty. -

Initial governance state.

Intake does not establish confirmation.

A newly received report may remain pending contextual assessment.

Vulnerability Identification

VMM shall identify the weakness or suspected weakness described by the

report.

Examples include:

-

Authorization bypass. -

Authentication failure. -

Injection. -

Sensitive data exposure. -

Insecure deserialization. -

Privilege escalation. -

Unsafe dependency behavior. -

Cryptographic misuse. -

Security misconfiguration. -

Insecure agent tool execution.

Identification shall preserve the affected software function or component

where known.

Vulnerability Recognition

VMM shall distinguish a recognized vulnerability candidate from a confirmed

vulnerability.

Recognition establishes that the report warrants governance attention.

Confirmation requires competent evidence supporting the vulnerability's

existence within the assessed scope.

Recognized Vulnerability Candidate ≠ Confirmed Vulnerability.

Vulnerability Confirmation

VMM shall establish whether available evidence supports the existence of

the reported weakness.

Confirmation may rely on:

-

Reproducible technical behavior. -

Competent code analysis. -

Verified configuration conditions. -

Reliable vendor evidence. -

Relevant dependency evidence. -

Targeted security testing. -

Operational observations.

Confirmation methods shall be appropriate to the vulnerability and affected

software.

A vulnerability may be confirmed without demonstrating successful exploitation in production.

Confirmed Vulnerability ≠ Confirmed Security Incident.

False-Positive Determination

VMM shall support competent determination that a reported finding does not represent the claimed vulnerability within the assessed scope.

A false-positive determination should preserve:

-

Original finding. -

Affected software identity. -

Assessment method. -

Supporting evidence. -

Relevant assumptions. -

Determination scope. -

Reassessment conditions.

A finding shall not be classified as false positive merely because

remediation is inconvenient or the scanner result is disputed.

Disputed Finding ≠ Demonstrated False Positive.

Duplicate Vulnerability Identification

VMM shall identify reports concerning the same underlying vulnerability

where appropriate.

Duplicate recognition should preserve:

-

Original report references. -

Shared technical cause. -

Affected artifacts. -

Distinct exposure conditions. -

Relevant treatment relationships.

Two reports may concern the same underlying weakness while affecting different software artifacts or configurations.

Duplicate administrative records shall not erase distinct exposure or

remediation obligations.

Vulnerability Classification

VMM shall classify confirmed or suspected vulnerabilities according to

relevant technical characteristics.

Classification may concern:

-

Weakness category. -

Affected security property. -

Vulnerable function. -

Attack surface. -

Required privileges. -

Relevant input conditions. -

Dependency origin. -

Configuration dependency.

Classification supports analysis and treatment.

It does not independently establish severity or exploitability.

Vulnerability Severity

VMM shall assess the seriousness of a vulnerability using competent

criteria.

Severity may consider:

-

Potential confidentiality impact. -

Potential integrity impact. -

Potential availability impact. -

Attack complexity. -

Required privileges. -

User interaction. -

Scope of affected functionality.

A recognized severity scoring framework may provide useful reference information.

VMM does not prescribe a single mandatory severity methodology.

Severity shall remain distinguishable from organizational treatment

priority.

Vulnerability Severity ≠ Remediation Priority.

Vulnerability Exploitability

VMM shall assess conditions affecting whether a vulnerability can be exploited.

Relevant factors may include:

-

Vulnerable code reachability. -

Required authentication. -

Required privileges. -

Attack surface accessibility. -

Exploit prerequisites. -

Configuration. -

Available attack techniques. -

Existing protective controls.

Exploitability shall be assessed within the relevant software context.

Theoretical Exploitability ≠ Demonstrated Application Exploitability.

Exploit Availability

VMM shall preserve information concerning available exploitation methods

where relevant.

Examples include:

-

Public exploit descriptions. -

Proof-of-concept code. -

Vendor advisories. -

Reproduced exploitation techniques. -

Reliable reports of active exploitation.

Exploit availability may materially affect urgency.

It does not independently establish that the governed software has been exploited.

Public Exploit Available ≠ Confirmed Local Exploitation.

Vulnerability Exposure

VMM shall assess vulnerability exposure using relevant SEAM findings.

Exposure may depend on:

-

Network accessibility. -

Enabled functionality. -

Dependency reachability. -

Trust boundaries. -

Authentication. -

Authorization. -

Runtime privileges. -

Operating environment.

A confirmed vulnerability may remain inaccessible under specified conditions.

Such a determination shall preserve its assumptions and limitations.

Protected Asset Impact

VMM shall identify protected assets potentially affected by a vulnerability.

Assets may include:

-

Personal data. -

Financial information. -

Credentials. -

Software artifacts. -

Critical operations. -

Administrative permissions. -

Service availability. -

Operational infrastructure.

Impact shall remain connected to the relevant security property and

exposure context.

Operational Consequence Assessment

VMM shall assess potential operational consequences of material

vulnerabilities.

Consequences may include:

-

Unauthorized transactions. -

Data disclosure. -

Service disruption. -

Privilege escalation. -

Loss of software integrity. -

Compromise of dependent services. -

Unauthorized agent actions.

Potential consequences shall remain distinguishable from observed

consequences.

Potential Vulnerability Impact ≠ Observed Operational Damage.

Vulnerability Prioritization

VMM shall establish contextual treatment priorities.

Prioritization may consider:

-

Vulnerability severity. -

Exploitability. -

Actual exposure. -

Protected asset significance. -

Operational criticality. -

Available exploitation evidence. -

Remediation feasibility. -

Relevant organizational obligations.

A critical severity score may warrant urgent action even when some exposure conditions remain uncertain.

A lower-scored vulnerability may warrant high priority where actual exposure and consequences are significant.

Severity Ranking ≠ Complete Treatment Priority Determination.

Vulnerability Ownership References

VMM shall preserve the responsible roles or mechanisms associated with

vulnerability assessment and treatment.

Relevant responsibilities may include:

-

Technical investigation. -

Remediation implementation. -

Security verification. -

Operational coordination. -

Treatment decision-making. -

Escalation.

Responsibility for remediation shall remain distinguishable from authority

to accept residual security risk.

Remediation Responsibility ≠ Residual Risk Acceptance Authority.

Vulnerability Treatment Governance

VMM shall establish identifiable treatment decisions.

Possible treatments include:

-

Corrective remediation. -

Dependency update. -

Configuration correction. -

Control modification. -

Compensating protection. -

Exposure reduction. -

Temporary restriction. -

Decommissioning of affected functionality. -

Authorized deferral.

Treatment shall remain connected to the affected vulnerability and software

state.

A selected treatment does not establish successful execution.

Remediation Planning

VMM shall preserve material remediation plans.

Plans may identify:

-

Affected software. -

Vulnerability reference. -

Intended corrective action. -

Relevant dependencies. -

Required verification. -

Responsible roles. -

Operational constraints. -

Expected treatment conditions.

A remediation plan shall not be represented as completed remediation.

Remediation Planned ≠ Remediation Implemented.

Corrective Action Correspondence

VMM shall preserve the relationship between the vulnerability and the

action intended to address it.

Corrective actions may include:

-

Source-code modification. -

Dependency replacement. -

Configuration correction. -

Permission restriction. -

Removal of vulnerable functionality. -

Security control improvement.

The corrective action shall be assessed against the underlying

vulnerability mechanism.

Software Change Completed ≠ Vulnerability Corrected.

Patch Implementation

VMM shall identify patches intended to correct vulnerabilities.

Relevant evidence may include:

-

Patch identity. -

Affected artifact. -

Code changes. -

Dependency changes. -

Build references. -

Configuration changes. -

Implementation records.

A patch being applied does not independently demonstrate that the vulnerability has been eliminated.

Patch Applied ≠ Remediation Verified.

Dependency Vulnerability Remediation

VMM shall govern corrective treatment of vulnerable dependencies.

Treatment may involve:

-

Updating the dependency. -

Replacing the component. -

Removing affected functionality. -

Restricting exposure. -

Applying a vendor correction. -

Implementing compensating controls.

Dependency remediation shall preserve actual artifact composition.

Updating a dependency manifest does not establish that the corrected component is present in the deployed artifact.

Dependency Manifest Updated ≠ Deployed Dependency Corrected.

Configuration Vulnerability Remediation

VMM shall govern corrective changes addressing security misconfiguration.

Examples include:

-

Disabling exposed debug interfaces. -

Restricting network access. -

Correcting authentication settings. -

Reducing excessive privileges. -

Protecting sensitive configuration values.

Remediation verification shall assess the configuration actually applied to

the relevant software environment.

Compensating Protection

VMM shall recognize protective measures intended to reduce vulnerability

exposure or consequences without necessarily removing the underlying weakness.

Examples include:

-

Network restrictions. -

Additional authorization checks. -

Feature isolation. -

Runtime monitoring. -

Temporary access restrictions.

Compensating protection shall remain distinguishable from elimination of

the vulnerability.

Compensating Control Active ≠ Underlying Vulnerability Removed.

STCM governs verification of the compensating control's effectiveness.

SRCM governs any relevant residual risk acceptance.

Temporary Mitigation

VMM shall preserve temporary measures used while permanent remediation

remains incomplete.

A temporary mitigation should identify:

-

The vulnerability. -

The affected software. -

The mitigation mechanism. -

Its applicable scope. -

Its expected duration. -

Verification evidence. -

Remaining limitations. -

Reassessment conditions.

Temporary mitigation shall not silently become permanent vulnerability

closure.

Vulnerability Deferral

VMM shall recognize decisions to defer vulnerability treatment.

Deferral may arise from:

-

Operational constraints. -

Dependency availability. -

Remediation complexity. -

Planned software replacement. -

Technical compatibility. -

Required maintenance windows.

Deferral shall preserve the unresolved vulnerability state.

It shall not independently establish authorized acceptance of residual

security risk.

Remediation Deferred ≠ Security Risk Accepted.

Vulnerability Exception References

VMM shall preserve exceptions relevant to vulnerability treatment.

An exception may concern:

-

Treatment timing. -

Temporary compensating controls. -

Operational restrictions. -

Conditional continuation. -

Alternative remediation.

VMM records the vulnerability-specific exception conditions.

SRCM governs residual risk evaluation and competent acceptance where applicable.

Vulnerability Exception Recorded ≠ Authorized Risk Acceptance.

Remediation Implementation Evidence

VMM shall preserve evidence that the intended corrective action was performed.

Evidence may include:

-

Source-code changes. -

Patch records. -

Dependency updates. -

Configuration changes. -

Build records. -

Deployment references. -

Operational change records.

Implementation evidence shall remain distinguishable from verification

evidence.

Corrective Action Evidence ≠ Verified Vulnerability Resolution.

Remediation Verification Purpose

VMM shall establish the purpose of remediation verification.

The purpose is to determine whether the corrective action sufficiently addresses the recognized vulnerability within the relevant software artifact, configuration, and operating conditions.

Verification shall remain connected to the original vulnerability mechanism.

A generic passing test shall not automatically establish remediation.

Remediation Verification Method

VMM shall identify competent methods for assessing corrective action

effectiveness.

Methods may include:

-

Reproduction of the original weakness. -

Targeted security testing. -

Code analysis. -

Configuration verification. -

Dependency composition verification. -

Runtime assessment. -

Relevant regression testing.

Verification method suitability shall be assessed against the vulnerability

and the corrective action.

STCM governs the underlying testing and control evidence.

VMM interprets that evidence for vulnerability resolution.

Remediation Artifact Correspondence

VMM shall preserve the identity of the software artifact against which

remediation is verified.

A successful test against a temporary build shall not establish that the intended release or production artifact is corrected.

Remediation Verified in Build A ≠ Remediation Verified in Build B.

Where a correction is intended for multiple affected artifacts, the relevant correspondence and verification conditions shall remain identifiable.

Remediation Configuration Correspondence

VMM shall assess whether verification applies to the relevant configuration.

A vulnerability may appear corrected in one environment while remaining exploitable under another configuration.

Relevant conditions may include:

-

Enabled features. -

Dependency settings. -

Runtime privileges. -

Authentication. -

Network accessibility. -

External integrations.

A favorable verification result shall remain bounded by its configuration.

Remediation Effectiveness

VMM shall determine whether competent evidence supports resolution of the

recognized vulnerability.

A favorable determination should establish:

-

Correct vulnerability identity. -

Relevant software artifact. -

Corrective action correspondence. -

Applicable configuration. -

Suitable verification method. -

Observed corrective behavior. -

Relevant evidence. -

Known limitations.

A remediation determination shall not exceed the supported scope.

Remediation Implementation ≠ Remediation Effectiveness.

Remediation Regression

VMM shall assess whether a corrective action introduces or reintroduces

material security weaknesses.

Relevant conditions may include:

-

Changed authorization behavior. -

Modified authentication logic. -

New dependency exposure. -

Changed privileges. -

New attack surfaces. -

Broken protective controls.

A vulnerability may be corrected while a separate vulnerability is introduced.

Original Vulnerability Resolved ≠ No New Security Weakness Introduced.

STCM provides relevant regression testing evidence.

Residual Vulnerability Conditions

VMM shall preserve vulnerability conditions that remain after treatment.

Examples include:

-

Partially corrected functionality. -

Remaining vulnerable configurations. -

Unpatched affected components. -

Incomplete deployment. -

Unverified remediation. -

Continuing exposure under specified conditions.

Residual vulnerability conditions shall remain distinguishable from

residual security risk acceptance.

Residual Vulnerability Identified ≠ Residual Risk Accepted.

Vulnerability Resolution Determination

VMM shall establish whether a vulnerability is sufficiently resolved within the assessed scope.

Possible determinations include:

-

Verified resolved. -

Partially resolved. -

Mitigated but unresolved. -

Not resolved. -

Verification pending. -

Evidence-limited. -

Reassessment required.

Resolution shall remain connected to the actual software artifact and

configuration.

A vulnerability shall not be considered verified resolved solely because a

corrective change was merged.

Vulnerability Closure Eligibility

VMM shall assess whether a vulnerability record is eligible for administrative closure.

Eligibility should consider:

-

Confirmation status. -

Treatment status. -

Remediation verification. -

Residual conditions. -

Evidence sufficiency. -

Relevant authority and workflow requirements.

Administrative closure shall not substitute for technical resolution.

Vulnerability Closure ≠ Verified Vulnerability Remediation.

Vulnerability Administrative Closure

VMM shall preserve the reason for administrative closure.

Possible reasons may include:

-

Verified remediation. -

Confirmed false positive. -

Duplicate record consolidation. -

Superseded software artifact. -

Other competently authorized administrative disposition.

The closure reason shall remain explicit.

A record closed because its software artifact was retired shall not be

represented as technically remediated.

Vulnerability Reopening

VMM shall support reopening vulnerability records when material evidence invalidates a previous resolution or closure determination.

Triggers may include:

-

Failed remediation verification. -

Reintroduced vulnerable code. -

Incorrect dependency update. -

New exploitability evidence. -

Production configuration differences. -

Contradictory test results. -

Incomplete deployment.

Reopening shall preserve historical records.

Previously Closed Vulnerability ≠ Permanently Resolved Vulnerability.

Vulnerability Evidence Governance

VMM shall preserve evidence supporting vulnerability determinations.

Evidence may include:

-

Scanner findings. -

Vendor advisories. -

Security test results. -

Code analysis. -

Runtime observations. -

Dependency composition. -

Configuration records. -

Patch records. -

Remediation verification results.

Evidence shall remain connected to the determination it supports.

Vulnerability Evidence Competence

VMM shall assess whether evidence is suitable for the relevant vulnerability claim.

Competence may depend on:

-

Correct artifact identity. -

Applicable configuration. -

Relevant vulnerability mechanism. -

Appropriate assessment method. -

Reliable observations. -

Temporal relevance. -

Known limitations.

A recognized scanner or advisory may provide useful evidence without independently establishing application-specific vulnerability status.

Vulnerability Evidence Integrity

VMM shall preserve material evidence integrity.

Relevant conditions may include:

-

Evidence origin. -

Record authenticity. -

Artifact correspondence. -

Assessment timing. -

Change history. -

Preservation of contradictory findings.

Evidence integrity shall remain distinguishable from evidence sufficiency.

Intact Vulnerability Report ≠ Sufficient Vulnerability Evidence.

Vulnerability Evidence Sufficiency

VMM shall assess whether available evidence supports the intended

vulnerability determination.

Evidence sufficient to identify a candidate may be insufficient to confirm exploitability.

Evidence sufficient to establish patch implementation may be insufficient to verify remediation.

Evidence sufficient for one artifact may be insufficient for another.

Evidence of Patch Implementation ≠ Evidence of Vulnerability Resolution.

Vulnerability Uncertainty

VMM shall preserve material uncertainty.

Uncertainty may concern:

-

Vulnerability applicability. -

Affected artifact identity. -

Exploitability. -

Exposure. -

Severity. -

Operational impact. -

Remediation completeness. -

Verification sufficiency. -

Residual conditions.

Unknown conditions shall not be silently converted into favorable

determinations.

Unknown Vulnerability Status ≠ No Vulnerability.

Vulnerability Evidence Conflict

A Vulnerability Evidence Conflict exists where material evidence or determinations are incompatible.

Examples include:

-

A scanner reports a vulnerability while competent analysis disputes

applicability. -

A patch report states remediation is complete while runtime testing demonstrates continued weakness. -

Dependency records indicate an updated component while the deployed

artifact contains an older version.

VMM shall preserve the conflict and relevant evidence.

A favorable administrative status shall not override contradictory

technical evidence.

Vulnerability Evidence Gap

A Vulnerability Evidence Gap exists where information necessary for competent vulnerability determination is missing.

Examples include:

-

Unknown affected artifact. -

Missing dependency composition. -

Unverified exploitability. -

Missing remediation evidence. -

Unavailable test results. -

Unknown production configuration. -

Incomplete deployment correspondence.

A gap shall remain identifiable until competently addressed.

Critical Vulnerability Governance Gap

A Critical Vulnerability Governance Gap exists where unresolved information prevents competent determination or treatment of a materially consequential vulnerability.

Examples include:

-

A critical privileged-access weakness without competent confirmation or

exclusion evidence. -

A reported vulnerability affecting production software whose artifact

identity cannot be established. -

A critical dependency vulnerability with unknown reachability and no

adequate assessment. -

A remediation marked complete without verification against the affected

production artifact. -

An autonomous agent closing critical vulnerability records without

preserving competent evidence.

A critical gap shall prevent unsupported favorable vulnerability

determinations.

It does not independently establish exploitation or operational damage.

Critical Vulnerability Governance Gap ≠ Confirmed Security Incident.

Vulnerability Governance States

VMM shall support identifiable vulnerability governance states.

Possible states include:

Reported: Vulnerability information has been received.

Identification-Pending: The affected weakness or software subject remains

insufficiently identified.

Candidate: A potentially relevant vulnerability has been recognized.

Applicability-Pending: Software-specific relevance remains unresolved.

Confirmation-Pending: Technical confirmation remains incomplete.

Confirmed: Competent evidence supports vulnerability existence.

False-Positive: Competent evidence supports exclusion of the reported

weakness within the assessed scope.

Duplicate: The report corresponds to an existing vulnerability governance

object.

Severity-Pending: Severity remains insufficiently established.

Exploitability-Pending: Exploitability remains unresolved.

Exposure-Pending: Relevant exposure conditions remain unresolved.

Prioritization-Pending: Treatment priority remains undetermined.

Treatment-Planned: A treatment approach has been established.

Remediation-Active: Corrective action is underway.

Mitigated: Exposure or consequences have been reduced without necessarily

eliminating the vulnerability.

Remediation-Implemented: The intended corrective action has been

performed.

Verification-Pending: Remediation effectiveness remains unverified.

Verification-Active: Remediation verification is underway.

Verified-Resolved: Competent evidence supports resolution within the assessed scope.

Partially-Resolved: Some affected conditions remain unresolved.

Unresolved: The vulnerability remains present or cannot be competently

resolved.

Deferred: Treatment has been postponed under identified conditions.

Exception-Referenced: A relevant exception exists without automatically

establishing risk acceptance.

Evidence-Limited: Material supporting evidence is insufficient.

Conflict-Detected: Material vulnerability evidence is incompatible.

Critical-Gap: A consequential governance condition remains unresolved.

Closure-Eligible: Closure conditions are sufficiently established.

Administratively-Closed: The record has been closed for an identified

reason.

Reopened: A previous closure or resolution determination requires renewed

treatment.

Reassessment-Required: Material changes require renewed evaluation.

Superseded: The determination has been replaced by a later governed

assessment.

These states shall preserve distinctions among vulnerability existence,

treatment, verification, and administrative disposition.

Vulnerability State ≠ Remediation State ≠ Administrative Closure State.

Vulnerability Lifecycle Continuity

VMM shall preserve vulnerability history across material software changes.

Continuity should support identification of:

-

Original vulnerability reports. -

Affected artifacts. -

Confirmation evidence. -

Severity assessments. -

Exploitability assessments. -

Exposure changes. -

Treatment decisions. -

Corrective actions. -

Verification results. -

Residual conditions. -

Closure reasons. -

Reopening events. -

Reassessment decisions.

A new vulnerability record shall not erase the history of an earlier

unresolved condition affecting the same software.

Vulnerability Reassessment

VMM shall support reassessment when material conditions change.

Triggers may include:

-

New exploit information. -

New software versions. -

Dependency changes. -

Configuration changes. -

New exposure evidence. -

Failed remediation. -

Changed operational context. -

Newly discovered affected components. -

Contradictory technical findings. -

Reintroduced vulnerable functionality.

Reassessment shall preserve relevant historical determinations.

Autonomous Vulnerability Management

VMM shall recognize vulnerability management performed or assisted by

autonomous systems.

Automated activities may include:

-

Vulnerability discovery. -

Advisory analysis. -

Dependency inspection. -

Severity estimation. -

Patch generation. -

Remediation testing. -

Ticket updates. -

Vulnerability reporting.

Automation may improve assessment and treatment capacity.

It does not independently establish evidence competence, technical resolution, or legitimate risk acceptance authority.

Autonomous Vulnerability Processing ≠ Competent Vulnerability Resolution.

Autonomous Remediation Authority

VMM shall preserve the authority boundary applicable to autonomous

corrective actions.

An agent may be authorized to:

-

Propose a patch. -

Modify development code. -

Update dependencies. -

Execute approved tests. -

Submit remediation evidence.

Such authority shall not automatically extend to:

-

Production deployment. -

Modification of security requirements. -

Acceptance of residual risk. -

Closure of critical vulnerability records without required verification. -

Disabling mandatory protective controls.

Authority to Implement Remediation ≠ Authority to Accept Residual Risk.

Autonomous Vulnerability Closure Control

VMM shall preserve the distinction between an agent updating a

vulnerability record and competent technical resolution.

An automated closure operation should preserve:

-

Vulnerability identity. -

Affected artifact. -

Corrective action. -

Verification evidence. -

Verification scope. -

Residual conditions. -

Closure reason. -

Applicable authorization.

An agent shall not convert a successful test or completed code change into

verified remediation without sufficient supporting evidence.

Governed Software Vulnerability State

The principal output of VMM is:

Governed Software Vulnerability State

The state shall preserve sufficient information to identify:

Vulnerability Governance Object: The governed vulnerability subject.

Vulnerability Identity: The stable vulnerability reference.

Report Origin: The source of the vulnerability information.

Report Intake: The initial recognition record.

Software Identity: The affected software object.

Artifact Identity: The relevant artifact and version.

Dependency Correspondence: Relevant component relationships.

Configuration: The applicable configuration conditions.

Security Requirements: Relevant SRQM references.

Exposure Conditions: Relevant SEAM references.

Testing Evidence: Relevant STCM references.

Vulnerability Description: The reported or established weakness.

Recognition State: The candidate or recognized status.

Confirmation State: The evidence-supported existence determination.

False-Positive Determination: Relevant exclusion evidence.

Duplicate References: Related vulnerability records.

Classification: The relevant weakness category.

Severity: The assessed seriousness.

Exploitability: The relevant exploitation conditions.

Exploit Availability: Relevant exploit information.

Exposure: The software-specific exposure context.

Protected Assets: The potentially affected assets.

Operational Consequences: Potential or observed effects.

Treatment Priority: The contextual priority.

Ownership References: Relevant responsibility references.

Treatment Decision: The selected treatment.

Remediation Plan: The intended corrective action.

Corrective Action: The performed software or configuration changes.

Patch Evidence: Relevant patch implementation records.

Dependency Remediation: Relevant dependency corrections.

Configuration Remediation: Relevant configuration corrections.

Compensating Protection: Applicable alternative protections.

Temporary Mitigation: Relevant interim measures.

Deferral: Relevant treatment postponement.

Exception References: Relevant exception conditions.

Remediation Evidence: The implementation evidence.

Verification Purpose: The intended verification objective.

Verification Method: The selected assessment method.

Verification Artifact: The artifact assessed.

Verification Configuration: The applicable configuration.

Verification Results: The observed corrective behavior.

Remediation Effectiveness: The supported resolution determination.

Regression Assessment: Relevant newly introduced weakness assessment.

Residual Vulnerability Conditions: Remaining unresolved conditions.

Resolution State: The recognized technical resolution status.

Closure Eligibility: The supported closure condition.

Administrative Closure: The closure reason and status.

Reopening: Relevant reopening events.

Evidence Competence: The suitability of supporting evidence.

Evidence Integrity: The integrity of material records.

Evidence Sufficiency: The supported determination boundaries.

Vulnerability Uncertainty: Material unresolved conditions.

Evidence Conflicts: Contradictory findings.

Evidence Gaps: Missing material information.

Critical Governance Gaps: Consequential unresolved conditions.

Vulnerability Governance State: The recognized overall condition.

Lifecycle Continuity: Historical vulnerability relationships.

Reassessment: Relevant reassessment records.

Autonomous Management References: Applicable automation activity.

Authority References: Relevant action and closure authorization

conditions.

Downstream References: Information required by SRCM.

The state shall not independently establish final software security

conformity, authorized residual risk acceptance, software release authorization, or deployment authorization.

Minimum Implementation Framework

An implementation of VMM shall provide at least six functional capabilities. Capability 1 — Vulnerability Identification & Intake

The implementation shall support identifiable vulnerability reports and

affected software references.

Minimum functions include:

-

Recording vulnerability origin. -

Establishing vulnerability identity. -

Preserving software artifact references. -

Recording dependency and configuration correspondence. -

Identifying reported weaknesses. -

Preserving initial evidence. -

Establishing initial governance states.

Minimum Output: Vulnerability Governance Object. Capability 2 — Vulnerability Confirmation & Contextual Assessment

The implementation shall support competent assessment of vulnerability

existence and applicability.

Minimum functions include:

-

Assessing report applicability. -

Evaluating confirmation evidence. -

Identifying false positives. -

Identifying duplicate reports. -

Classifying vulnerabilities. -

Assessing exploitability. -

Associating SEAM exposure conditions.

Minimum Output: Governed Vulnerability Assessment State. Capability 3 — Severity, Impact & Treatment Prioritization

The implementation shall support contextual vulnerability treatment

decisions.

Minimum functions include:

-

Recording severity. -

Assessing exploitability. -

Identifying protected assets. -

Assessing potential operational consequences. -

Preserving exploit availability references. -

Establishing treatment priority. -

Preserving responsibility references.

Minimum Output: Vulnerability Treatment Priority State. Capability 4 — Vulnerability Treatment & Remediation Control

The implementation shall support traceable corrective actions.

Minimum functions include:

-

Recording treatment decisions. -

Preserving remediation plans. -

Associating corrective changes. -

Recording patch implementation. -

Preserving dependency and configuration remediation. -

Identifying compensating controls. -

Recording deferrals and exception references.

Minimum Output: Governed Vulnerability Treatment State. Capability 5 — Remediation Verification & Closure Integrity

The implementation shall support competent technical resolution

determinations.

Minimum functions include:

-

Identifying verification objectives. -

Preserving artifact and configuration correspondence. -

Recording suitable verification methods. -

Associating STCM testing evidence. -

Evaluating remediation effectiveness. -

Preserving residual vulnerability conditions. -

Distinguishing technical resolution from administrative closure.

Minimum Output: Governed Vulnerability Remediation Verification State. Capability 6 — Vulnerability State, Continuity & Downstream Handoff

The implementation shall consolidate vulnerability governance information.

Minimum functions include:

-

Establishing vulnerability governance states. -

Preserving uncertainty and evidence conflicts. -

Identifying critical governance gaps. -

Maintaining vulnerability history. -

Supporting reopening and reassessment. -

Preserving autonomous action authority references. -

Providing SRCM with traceable vulnerability information.

Minimum Output: Governed Software Vulnerability State.

These capabilities define methodological implementation requirements.

They do not prescribe a particular vulnerability management platform,

scanning tool, ticketing system, severity scoring framework, or execution engine.

GIE™ Compatibility

GIE-Compatible: Yes

VMM may provide structured governance inputs to the Governance Intelligence Engine™ (GIE).

Potential GIE-supported functions include:

-

Vulnerability report classification. -

Vulnerability applicability analysis. -

Artifact and dependency correspondence assessment. -

Confirmation evidence evaluation. -

Duplicate finding identification. -

Severity and exploitability comparison. -

Exposure-aware treatment prioritization. -

Remediation action traceability. -

Remediation verification evidence assessment. -

Vulnerability state consistency checking. -

Administrative closure integrity analysis. -

Residual vulnerability condition detection. -

Critical evidence gap identification. -

Vulnerability reopening trigger detection. -

Autonomous remediation authority analysis. -

Downstream security conformity handoff preparation.

GIE™ may assist with structured analysis, evidence organization, inconsistency detection, and methodological completeness evaluation.

GIE™ shall not independently convert reported findings into confirmed

vulnerabilities, implemented patches into verified remediation, or closed tickets into authorized residual risk acceptance.

The governing principle remains:

OOF® logic adapts to the governed reality. The engine adapts to OOF®

logic.

Governance Requirements

A conforming VMM implementation shall satisfy the following requirements.

VMM-R01 — Vulnerability Identity: Material vulnerability determinations

shall reference identifiable vulnerability objects.

VMM-R02 — Software Correspondence: Vulnerability findings shall remain

connected to affected software.

VMM-R03 — Artifact Correspondence: Vulnerability determinations shall

preserve relevant artifact identity.

VMM-R04 — Dependency Correspondence: Relevant dependency relationships

shall remain traceable.

VMM-R05 — Configuration Correspondence: Material configuration conditions

shall remain identifiable.

VMM-R06 — Report Origin: Vulnerability information shall preserve its

source.

VMM-R07 — Intake: Material vulnerability reports shall remain

identifiable from initial recognition.

VMM-R08 — Identification: Reported weaknesses shall be sufficiently

described.

VMM-R09 — Recognition: Vulnerability candidates shall remain

distinguishable from confirmed vulnerabilities.

VMM-R10 — Confirmation: Confirmation shall require competent supporting

evidence.

VMM-R11 — False Positives: False-positive determinations shall preserve

their evidentiary basis.

VMM-R12 — Duplicates: Duplicate recognition shall not erase distinct

affected conditions.

VMM-R13 — Classification: Vulnerabilities shall preserve relevant

technical classifications.

VMM-R14 — Severity: Severity shall be assessed using competent criteria.

VMM-R15 — Exploitability: Exploitability shall be evaluated in the

relevant software context.

VMM-R16 — Exploit Availability: Relevant exploit information shall remain

identifiable.

VMM-R17 — Exposure: Vulnerability exposure shall remain connected to

relevant SEAM findings.

VMM-R18 — Protected Assets: Material affected assets shall remain

identifiable.

VMM-R19 — Operational Consequences: Potential and observed consequences

shall remain distinguishable.

VMM-R20 — Prioritization: Treatment priority shall consider relevant

contextual conditions.

VMM-R21 — Ownership: Material treatment responsibilities shall remain

identifiable.

VMM-R22 — Treatment: Vulnerability treatment decisions shall remain

traceable.

VMM-R23 — Remediation Planning: Intended corrective actions shall remain

identifiable.

VMM-R24 — Corrective Correspondence: Remediation actions shall remain

connected to the vulnerability addressed.

VMM-R25 — Patch Implementation: Patch execution shall remain

distinguishable from verified remediation.

VMM-R26 — Dependency Remediation: Dependency corrections shall preserve

actual artifact correspondence.

VMM-R27 — Configuration Remediation: Corrective configuration shall

remain verifiable.

VMM-R28 — Compensating Protection: Alternative protections shall preserve

their limitations.

VMM-R29 — Temporary Mitigation: Interim treatment shall not be

represented as permanent resolution.

VMM-R30 — Deferral: Deferred vulnerabilities shall remain identifiable as

unresolved where applicable.

VMM-R31 — Exceptions: Exception records shall remain distinct from

authorized risk acceptance.

VMM-R32 — Remediation Evidence: Corrective action evidence shall remain

traceable.

VMM-R33 — Verification Purpose: Remediation verification shall address

the recognized vulnerability.

VMM-R34 — Verification Method: Verification methods shall be suitable for

the intended resolution claim.

VMM-R35 — Verification Artifact: Remediation evidence shall correspond to

the relevant artifact.

VMM-R36 — Verification Configuration: Verification shall preserve

relevant configuration conditions.

VMM-R37 — Remediation Effectiveness: Resolution shall require competent

supporting evidence.

VMM-R38 — Regression: Corrective changes shall support relevant

regression assessment.

VMM-R39 — Residual Conditions: Remaining vulnerability conditions shall

remain explicit.

VMM-R40 — Resolution: Technical resolution shall remain distinguishable

from administrative closure.

VMM-R41 — Closure Eligibility: Closure shall require an identifiable

competent basis.

VMM-R42 — Administrative Closure: Closure reasons shall remain explicit.

VMM-R43 — Reopening: Invalidated resolution or closure determinations

shall support reopening.

VMM-R44 — Evidence Competence: Vulnerability determinations shall rely on

suitable evidence.

VMM-R45 — Evidence Integrity: Material evidence integrity shall remain

preserved.

VMM-R46 — Evidence Sufficiency: Evidence shall support the specific

determination claimed.

VMM-R47 — Uncertainty: Material uncertainty shall remain explicit.

VMM-R48 — Evidence Conflicts: Contradictory vulnerability evidence shall

remain visible.

VMM-R49 — Evidence Gaps: Missing evidence shall not be treated as

demonstrated remediation.

VMM-R50 — Critical Gaps: Consequential unresolved conditions shall

prevent unsupported favorable determinations.

VMM-R51 — Governance States: Vulnerability, treatment, verification, and

closure states shall remain distinguishable.

VMM-R52 — Lifecycle Continuity: Vulnerability history shall remain

traceable across material changes.

VMM-R53 — Reassessment: Material changes shall trigger competent

reassessment where applicable.

VMM-R54 — Autonomous Management: Automated vulnerability actions shall

remain subject to the same evidence requirements.

VMM-R55 — Autonomous Authority: Remediation execution authority shall

remain distinct from residual risk acceptance authority.

VMM-R56 — Autonomous Closure: Automated closure shall not replace

competent remediation verification.

VMM-R57 — Governance Output: The implementation shall produce an

identifiable Governed Software Vulnerability State.

VMM-R58 — Downstream Boundary: Vulnerability management shall not

independently establish final security conformity or residual risk acceptance.

Governance Logic

The canonical VMM governance logic is:

Receive Governed Software Security Requirements State

↓

Receive Governed Software Security Exposure State

↓

Receive Governed Software Security Testing & Control State

↓

Identify Vulnerability Governance Object

↓

Establish Software Artifact and Configuration Correspondence

↓

Record Vulnerability Report Origin

↓

Recognize Vulnerability Candidate

↓

Assess Vulnerability Applicability

↓

Evaluate Confirmation Evidence

↓

Determine Vulnerability Confirmation State

↓

Identify False Positives and Duplicate Reports

↓

Classify Vulnerability

↓

Assess Severity

↓

Assess Exploitability

↓

Assess Relevant Exposure

↓

Identify Protected Assets and Operational Consequences

↓

Determine Treatment Priority

↓

Identify Responsible Treatment Roles

↓

Establish Vulnerability Treatment Decision

↓

Define Remediation Plan

↓

Implement Corrective Action

↓

Preserve Patch, Dependency, or Configuration Evidence

↓

Identify Compensating Controls and Temporary Mitigations

↓

Establish Remediation Verification Purpose

↓

Select Competent Verification Method

↓

Verify Relevant Artifact and Configuration

↓

Evaluate Remediation Effectiveness

↓

Assess Security Regression

↓

Identify Residual Vulnerability Conditions

↓

Determine Vulnerability Resolution State

↓

Assess Administrative Closure Eligibility

↓

Preserve Uncertainty, Conflicts, and Evidence Gaps

↓

Determine Vulnerability Governance State

↓

Establish Governed Software Vulnerability State

↓

Handoff to Security Risk & Conformity

↓

Preserve Vulnerability Lifecycle Continuity

↓

Reassess or Reopen When Material Conditions Change

The principal output is:

Governed Software Vulnerability State

Governance Outputs

VMM may produce:

Vulnerability Governance Object

Vulnerability Management Purpose Record

Vulnerability Management Scope Record

Software Vulnerability Artifact Correspondence Record

Dependency Vulnerability Correspondence Record

Configuration Vulnerability Correspondence Record

Vulnerability Report Origin Record

Vulnerability Intake Record

Vulnerability Identification Record

Vulnerability Candidate Record

Vulnerability Recognition State

Vulnerability Confirmation Assessment

False-Positive Determination

Duplicate Vulnerability Reference

Vulnerability Classification Record

Vulnerability Severity Assessment

Vulnerability Exploitability Assessment

Exploit Availability Reference

Vulnerability Exposure Assessment

Protected Asset Impact Assessment

Operational Consequence Assessment

Vulnerability Treatment Priority State

Vulnerability Ownership Reference

Vulnerability Treatment Decision

Remediation Plan

Corrective Action Record

Patch Implementation Record

Dependency Remediation Record

Configuration Remediation Record

Compensating Protection Record

Temporary Mitigation Record

Vulnerability Deferral Record

Vulnerability Exception Reference

Remediation Implementation Evidence Record

Remediation Verification Specification

Remediation Verification Method Record

Remediation Artifact Correspondence Record

Remediation Configuration Correspondence Record

Remediation Effectiveness Assessment

Remediation Regression Assessment

Residual Vulnerability Condition Record

Vulnerability Resolution Determination

Vulnerability Closure Eligibility Assessment

Administrative Closure Record

Vulnerability Reopening Record

Vulnerability Evidence Register

Vulnerability Evidence Competence Assessment

Vulnerability Evidence Integrity Assessment

Vulnerability Evidence Sufficiency Assessment

Vulnerability Uncertainty State

Vulnerability Evidence Conflict

Vulnerability Evidence Gap

Critical Vulnerability Governance Gap

Vulnerability Governance State

Vulnerability Lifecycle Continuity Record

Vulnerability Reassessment Record

Autonomous Vulnerability Management Record

Autonomous Remediation Authority Reference

Autonomous Vulnerability Closure Assessment

SRCM Vulnerability Handoff Record

The principal output is:

Governed Software Vulnerability State

Critical Distinctions

VMM preserves the following distinctions.

Vulnerability Closure ≠ Verified Vulnerability Remediation

Vulnerability Report ≠ Confirmed Software Vulnerability

Vulnerability Management Activity ≠ Vulnerability Resolution

Component Vulnerability ≠ Universal Application Exploitability

Vulnerability State of Build A ≠ Vulnerability State of Build B

Vulnerability Presence ≠ Exploitability Under Every Configuration

Trusted Vulnerability Source ≠ Automatically Applicable Finding

Recognized Vulnerability Candidate ≠ Confirmed Vulnerability

Confirmed Vulnerability ≠ Confirmed Security Incident

Disputed Finding ≠ Demonstrated False Positive

Vulnerability Severity ≠ Remediation Priority

Theoretical Exploitability ≠ Demonstrated Application Exploitability

Public Exploit Available ≠ Confirmed Local Exploitation

Potential Vulnerability Impact ≠ Observed Operational Damage

Severity Ranking ≠ Complete Treatment Priority Determination

Remediation Responsibility ≠ Residual Risk Acceptance Authority

Remediation Planned ≠ Remediation Implemented

Software Change Completed ≠ Vulnerability Corrected

Patch Applied ≠ Remediation Verified

Dependency Manifest Updated ≠ Deployed Dependency Corrected

Compensating Control Active ≠ Underlying Vulnerability Removed

Remediation Deferred ≠ Security Risk Accepted

Vulnerability Exception Recorded ≠ Authorized Risk Acceptance

Corrective Action Evidence ≠ Verified Vulnerability Resolution

Remediation Verified in Build A ≠ Remediation Verified in Build B

Remediation Implementation ≠ Remediation Effectiveness

Original Vulnerability Resolved ≠ No New Security Weakness Introduced

Residual Vulnerability Identified ≠ Residual Risk Accepted

Previously Closed Vulnerability ≠ Permanently Resolved Vulnerability

Intact Vulnerability Report ≠ Sufficient Vulnerability Evidence

Evidence of Patch Implementation ≠ Evidence of Vulnerability Resolution

Unknown Vulnerability Status ≠ No Vulnerability

Critical Vulnerability Governance Gap ≠ Confirmed Security Incident

Vulnerability State ≠ Remediation State ≠ Administrative Closure State

Autonomous Vulnerability Processing ≠ Competent Vulnerability Resolution

Authority to Implement Remediation ≠ Authority to Accept Residual Risk

Cross-Architecture Boundary

VMM governs software vulnerability identification, confirmation,

classification, severity, exploitability, exposure, prioritization, treatment, remediation verification, closure integrity, and lifecycle continuity.

It does not independently establish:

-

Security requirement authority. -

Complete software threat exposure. -

General security control effectiveness. -

Final software security conformity. -

Authorized residual security risk acceptance. -

Software release authorization. -

Software deployment authorization. -

Security incident attribution. -

Legal liability.

Within SSS:

SRQM — Security Requirements Module establishes governed security

requirements.

SEAM — Security Exposure Assessment Module establishes governed exposure

conditions.

STCM — Security Testing & Control Module establishes governed security

testing and control conditions.

VMM — Vulnerability Management Module establishes governed vulnerability

conditions.

SRCM — Security Risk & Conformity Module governs security conformity,

residual risk, and security acceptance.

Within SLGA™:

SRS governs software requirements.

SCS governs software creation.

SDS governs software dependencies.

SCFS governs software configuration.

SQS governs software quality.

SSS governs software security.

SRLS governs software release.

SDPS governs software deployment.

SCMS governs software changes.

SRTS governs software retirement.

SLCS governs cross-cutting lifecycle control.

VMM preserves vulnerability-specific references to relevant OOF® authority,

accountability, operational reality, identity, and autonomous-agent governance mechanisms without replacing their competent governance domains.

Canonical Boundary

VMM begins when a reported, suspected, detected, or previously recognized software vulnerability requires competent governance assessment.

VMM governs:

-

Vulnerability identity. -

Report origin. -

Intake. -

Software correspondence. -

Artifact correspondence. -

Dependency correspondence. -

Configuration correspondence. -

Vulnerability identification. -

Candidate recognition. -

Applicability. -

Confirmation. -

False-positive determination. -

Duplicate identification. -

Classification. -

Severity. -

Exploitability. -

Exploit availability. -

Exposure. -

Protected asset impact. -

Operational consequences. -

Prioritization. -

Ownership references. -

Treatment. -

Remediation planning. -

Corrective actions. -

Patch implementation. -

Dependency remediation. -

Configuration remediation. -

Compensating protection. -

Temporary mitigation. -

Deferral. -

Exception references. -

Remediation evidence. -

Verification purpose. -

Verification methods. -

Verification artifact correspondence. -

Verification configuration correspondence. -

Remediation effectiveness. -

Regression. -

Residual vulnerability conditions. -

Resolution. -

Closure eligibility. -

Administrative closure. -

Reopening. -

Evidence competence. -

Evidence integrity. -

Evidence sufficiency. -

Uncertainty. -

Evidence conflicts. -

Evidence gaps. -

Critical governance gaps. -

Vulnerability governance states. -

Lifecycle continuity. -

Reassessment. -

Autonomous vulnerability management. -

Autonomous remediation authority. -

Autonomous closure control.

VMM does not independently determine:

-

Final software security conformity. -

Residual security risk acceptance. -

Release authorization. -

Deployment authorization. -

Complete security assurance. -

Security incident attribution. -

Legal compliance certification. -

Legal liability.

VMM ends when the relevant vulnerability conditions have been represented as a:

Governed Software Vulnerability State

with sufficient vulnerability identity, software correspondence,

confirmation evidence, severity, exploitability, exposure, treatment, remediation verification, residual conditions, authority references, and lifecycle continuity for competent downstream security governance.

• Use Case 1 — A Dependency Vulnerability Is Marked Resolved, but the

Production Artifact Still Contains the Vulnerable Component

A software company maintains a cloud-based enterprise application.

A vulnerability scanner identifies a critical vulnerability in a

third-party dependency.

The development team updates the dependency manifest to a corrected version.

The build pipeline completes successfully.

The vulnerability management dashboard automatically changes the record to:

Status: Resolved

Remediation: Completed

Critical Vulnerabilities: 0

However, the production artifact was created using a cached dependency

layer.

The deployed software still contains the vulnerable component.

Under VMM, the assessment identifies:

Vulnerability Origin: Dependency security advisory.

Affected Software: Enterprise cloud application.

Vulnerability Confirmation: The affected dependency version is present in the deployed artifact.

Corrective Action: Dependency manifest updated.

Remediation Implementation: A change was performed in the development

source.

Artifact Correspondence: The deployed artifact does not contain the corrected dependency.

Verification Evidence: Insufficient for the deployed software.

Resolution State: Not Verified Resolved.

Administrative Closure: Unsupported by the available technical evidence.

The dependency manifest update does not establish that the deployed software has been corrected.

The relevant artifact must be identified and assessed.

The resulting distinctions are:

Dependency Manifest Updated ≠ Deployed Dependency Corrected.

Patch Applied ≠ Remediation Verified.

Vulnerability Closure ≠ Verified Vulnerability Remediation.

VMM preserves the unresolved vulnerability state and provides the relevant

information to SRCM.

Any residual risk acceptance requires a separate competent decision.

Use Case 2 — An Autonomous Security Agent Closes a Critical

Authorization Vulnerability Without Competent Verification

An organization operates an autonomous software security agent.

The agent is authorized to inspect code, propose patches, execute development tests, and update vulnerability records.

A critical authorization weakness is identified in an administrative API.

The weakness may allow an authenticated ordinary user to invoke a privileged operation.

The agent:

1.

Identifies the vulnerability. 2.

Generates a corrective patch. 3.

Modifies the authorization logic. 4.

Executes automated tests. 5.

Receives passing results. 6.

Closes the vulnerability ticket. 7.

Reports the issue as fully resolved.

A subsequent review discovers that the tests verified successful administrative access but did not test prohibited access by ordinary users.

The agent also lacks authority to accept residual security risk.

Under VMM, the assessment identifies:

Vulnerability: Privileged authorization enforcement weakness.

Affected Asset: Administrative operations.

Corrective Action: Modified authorization logic.

Remediation Implementation: Patch implemented.

Verification Scope: Positive administrative access tests.

Verification Gap: Missing negative authorization testing.

Remediation Effectiveness: Not sufficiently demonstrated.

Closure Eligibility: Not established.

Autonomous Authority: Authorized to propose and test remediation, but not

to accept residual risk.

Governance State: Verification Pending / Evidence-Limited.

The successful tests demonstrate only the behavior they actually assessed.

They do not establish that ordinary users are prevented from executing privileged operations.

The resulting distinctions are:

Test Execution Success ≠ Verified Vulnerability Resolution.

Administrative Closure ≠ Technical Remediation.

Authority to Implement Remediation ≠ Authority to Accept Residual Risk.

VMM preserves the vulnerability record, verification gap, and relevant

authority boundary.

The agent may perform additional authorized testing or request competent review.

It shall not silently convert incomplete verification into a favorable

vulnerability resolution determination.

Canonical Closing Statement

Software vulnerabilities do not become resolved merely because they have

been detected, classified, assigned, patched, or marked as closed.

A scanner may report a vulnerability that is not applicable to the software.

A confirmed vulnerability may require contextual assessment before its exploitability and operational significance are understood.

A critical severity score may not fully determine treatment priority.

A patch may be implemented without correcting the relevant deployed artifact.

A compensating control may reduce exposure without eliminating the underlying weakness.

And a vulnerability ticket may be closed while material verification evidence remains missing.

Vulnerability Report ≠ Confirmed Software Vulnerability.

Vulnerability Severity ≠ Remediation Priority.

Patch Applied ≠ Remediation Verified.

Compensating Control Active ≠ Underlying Vulnerability Removed.

Vulnerability Closure ≠ Verified Vulnerability Remediation.

These distinctions become increasingly important as automated vulnerability

management and autonomous software remediation become more common.

An AI agent may identify a vulnerability, generate a patch, execute tests, and update a dashboard within minutes.

That speed can be valuable.

But automated execution does not eliminate the need to establish what software was affected, whether the vulnerability was correctly understood, whether the corrective action addressed the underlying weakness, whether the intended artifact was verified, and which residual conditions remain.

Nor does remediation authority automatically establish authority to accept residual security risk.

VMM therefore governs the complete evidentiary relationship between

vulnerability identification, technical confirmation, contextual assessment, corrective action, verification, and resolution.

It does not promise that every vulnerability will be discovered.

It does not treat administrative closure as proof of security.

Its purpose is to ensure that vulnerability determinations remain connected to the software, evidence, and operating conditions they actually concern.

VMM transforms vulnerability reports, security findings, technical

assessments, corrective actions, remediation tests, and administrative records into a governed software vulnerability state — establishing which weaknesses are confirmed, how they affect the software, what treatment has occurred, what remediation has been competently verified, which conditions remain unresolved, and how those determinations continue across the software lifecycle.

Software vulnerability management becomes governable when vulnerability existence, contextual significance, treatment decisions, corrective actions, verification evidence, residual conditions, and administrative closure remain distinct, traceable, and connected to the actual software artifact and configuration throughout its lifecycle.